Understanding AWS Provider Dependency Updates in Infrastructure-As-Code: Empirical Study, Taxonomy, and Insights

Mahi Begoug, Ali Ouni, and Jasem Khelifi

***Abstract*—Infrastructure-as-Code (IaC) automates the configuration of cloud platforms through code. As business needs evolve, IaC files often become complex, containing hundreds of lines and multiple dependencies. These configurations rely on third-party providers to provision system infrastructure. Practitioners regularly update IaC code to align with evolving cloud provider specifications (i.e., AWS, GCP, Azure) and to address security issues or defects. Although prior work highlights the risks of outdated dependencies, it remains unclear whether IaC practitioners consistently update provider dependencies in accordance with official releases. To address this gap, we conduct a mixed-method empirical study focused on the Amazon Web Services (AWS) provider, one of the most widely used providers for provisioning cloud infrastructures. We analyze 23,404 Terraform (TF) related commits from 194 open-source TF projects, focusing on: (i) technical lag, which captures how long AWS provider dependencies remain unchanged in code; (ii) the frequency of dependency updates; (iii) the code review effort involved in updating AWS provider dependencies; and (iv) the motivations behind such updates. Our findings reveal that Terraform developers frequently rely on outdated provider versions, with the technical lag increasing steadily from 2017 to early 2025, reaching a monthly average of approximately 9 months by 2025. Quantitative analysis reveals that only 1.86% of TF-related commits involve updates to AWS provider dependencies, indicating that such updates are not a priority. Moreover, related code reviews are substantial, affecting a median of 7 files across multiple directories. Through thematic analysis, we identify nine key motivations for updating the AWS provider dependencies, with the top three being: *Providers Dependency Management*, *Terraform Compatibility Management*, and *Security Management*. These insights highlight a clear need for better support and tooling to help practitioners manage provider updates more effectively, minimizing disruption while modernizing infrastructure. We recommend adopting automated dependency management tools and improved update workflows to reduce technical lag and lower the cost of staying up to date.**

***Index Terms*—Infrastructure-as-code, amazon web services provider, infrastructure dependencies.**

I. Introduction

INFRASTRUCTURE-AS-CODE (IaC) has emerged as a modern approach to provisioning and managing modern software infrastructure through machine-readable configuration files [1]. By incorporating software engineering practices such as version control and automated testing, IaC enables more reliable and efficient infrastructure management. Several IaC tools have gained traction, including Terraform (TF), CloudFormation, and Ansible [2], [3]. While TF and CloudFormation are widely adopted for declarative infrastructure provisioning, Ansible, traditionally considered a configuration management tool, is often used to automate post-provisioning tasks within IaC workflows [4].

Among these, TF stands out as the most popular IaC tool, as confirmed by recent reports from GitHub [5] and Stack Overflow.[1] Developed by HashiCorp, Terraform is designed to manage and secure infrastructure resources across multi-cloud environments [6], while ensuring repeatable, idempotent operations [1]. Its key advantages include modularity, code reusability, version control, scalability, and automation [6].

To provision infrastructure with Terraform, practitioners use third-party providers through dependencies.[2] Terraform supports multiple cloud providers, including Amazon Web Services (AWS), Azure, Google Cloud, and others, allowing organizations to manage infrastructure across different clouds using a single tool. These providers allow Terraform to interact with cloud platforms via Application Programming Interfaces (APIs). HashiCorp reported that the AWS provider is the most used, with 4 billion downloads in 2025 [7], highlighting the widespread use of AWS infrastructures. However, in a fast-changing world, cloud-based infrastructure providers must update their products and releases frequently to meet user needs on time [1]. For example, AWS frequently releases new versions to fix bugs and vulnerabilities, add features, improve performance, and so on [1]. To keep its cloud services up-to-date, AWS has released an average of 4.52 monthly provider versions since 2017, continuing to 2024.[3]

When a new update becomes available to a provider, TF practitioners may consider upgrading to take advantage of the latest release, as it often includes bug fixes and performance improvements [1]. However, the latest release might require substantial migration effort from the previous configuration,

Received 11 April 2025; revised 30 July 2025; accepted 11 August 2025. Date of publication 20 August 2025; date of current version 11 December 2025. This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) under Grant RGPIN-2025-05416. *(Corresponding author: Mahi Begoug.)*

The authors are with the Department of Software Engineering and IT, ETS Montreal, University of Quebec, Montreal, QC H3C3J7, Canada (e-mail: mahi.begoug.1@ens.etsmtl.ca; ali.ouni@etsmtl.ca; jasem.khelifi.1@ens.etsmtl.ca).

Digital Object Identifier 10.1109/TSC.2025.3601051

[1] https://tinyurl.com/y6w9ss4j
[2] https://tinyurl.com/33bsz53b
[3] https://tinyurl.com/ncajv4vs

1939-1374 © 2025 Crown Copyright

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

making it time-consuming and error-prone [8], [9]. In addition, practitioners may revert to an earlier release to ensure infrastructure stability and avoid issues related to breaking changes or backward compatibility [1]. These practical considerations point to several challenges that remain underexplored in the IaC ecosystem. Specifically: (i) there is limited understanding of how long outdated provider dependencies persist in real-world client projects (i.e., technical lag); (ii) it is unclear how frequently such updates are performed in practice; (iii) there is a lack of empirical evidence on the engineering effort required to carry out these updates; and (iv) the underlying motivations that drive developers to update provider dependencies remain especially undocumented.

To better understand these challenges, we aim to investigate the practices for updating provider dependencies in IaC configuration, focusing on AWS provider dependencies in Terraform files. To that end, we conduct a mixed quantitative and qualitative study on 194 Terraform-based projects collected from GitHub, using at least the AWS provider dependency. From these projects, we analyze 23,404 TF related commits (i.e., at least one changed TF file) that span from 2017 to the start of 2025, leading to addressing the following research questions:

(RQ1) How long is the technical lag in AWS provider dependencies in Terraform-based projects?

Results: This RQ1 investigates the evolution of technical lag to assess the time typically required by Terraform developers to update the version of their AWS provider dependencies. Our results show a steady increase in technical lag from 2017 to 2025, reaching an average of approximately 9 months by 2025. A similar trend is observed for other providers, including Google Cloud Platform (GCP), Azure, and Kubernetes, particularly between 2022 and 2025. These findings suggest a growing delay in adopting newer provider versions across major platforms.

(RQ2) How frequently are AWS provider dependencies updated in Terraform-based projects ?

Results: RQ2 examines the update frequency to assess how often AWS provider dependencies are updated in practice. Our results show that, on average, only 1.86% of commits involving TF files include updates to AWS provider dependencies. This suggests that TF developers do not consistently prioritize updating these dependencies, performing such updates only occasionally. Similarly, we observe that dependencies for co-used providers such as Google (1.82%) and Azure (1.48%) exhibit comparable update frequencies.

(RQ3) What is the code review effort in AWS dependency updates?

Results: RQ3 analyzes the characteristics of pull requests, including AWS provider dependency updates. Our findings show that these pull requests differ significantly from those without such updates regarding the number of changed files (i.e., 7 changed files in median), code churn, number of commits, changed directories, and affected subsystems, with at least small effect sizes. This indicates that updating AWS provider dependencies tends to be associated with larger, more complex, and more review-intensive pull requests, implying a higher review effort. These findings also suggest that accumulated technical lag contributes to this increased effort. As the delay in updating grows, the required changes tend to span more files and subsystems, adding cognitive burden to the development workflow. As a result, Terraform practitioners may face substantial integration complexity, an increased risk of breaking changes, and missed opportunities to leverage performance, security, or compatibility improvements.

(RQ4) What are the developer's motivations when updating their AWS provider dependencies?

Results: We identify nine key motivations that drive Terraform developers to update their AWS provider dependencies, including: *Provider Dependency Management* (33.3%), *Terraform Compatibility Management* (31.5%), *Security Management* (7.9%), *Bug Fixes* (6.4%), *Terraform Code Refactoring* (5.9%), *Infrastructure Deployment* (5.4%), *Resource Management* (4.4%), *Monitoring* (2.8%), *Performance Improvement* (2.3%). These findings suggest that updating AWS provider dependencies is a non-trivial task, as developers primarily focus on managing complex provider relationships, maintaining compatibility with Terraform core versions, and preserving the reliability and stability of their infrastructure.

This paper extends our previous work published at the International Conference on Service-Oriented Computing (ICSOC) [10]. Specifically, this paper extends our prior work in the following ways:

- *Expanded dataset:* We significantly extend the dataset by analyzing an additional 165 real-world Terraform projects, selected to ensure diversity in development activity. In total, our empirical study now includes 194 open-source projects;
- *Quantitative analysis of dependency update practices:* We investigate the *technical lag* and *update frequency* of AWS provider dependencies to characterize how swiftly Terraform developers adopt new versions;
- *Code review effort assessment:* We examine the characteristics of code review practices associated with AWS provider updates to better understand the effort and complexity involved in managing such changes;
- *Qualitative taxonomy of update motivations:* We conduct a qualitative study to uncover the motivations behind dependency update practices, resulting in a taxonomy that complements and triangulates our quantitative findings.

Open Science: We provide a comprehensive package with datasets and scripts to support replication [11].

II. Background and Motivation

A. Terraform

Terraform (TF): is a vendor-agnostic IaC framework that abstracts low-level infrastructure management through a unified, code-based interface [1], [12]. It uses the declarative HashiCorp Configuration Language (HCL) to define the desired infrastructure state. During compilation, TF Core processes the configurations in a state file, leveraging provider plugins to manage resources across cloud platforms [12].

Configuration of TF providers: In TF, developers must declare the required providers to allow *TF Core* to install and use them. These declarations are typically written in ". *tf*" files,

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

Listing 1: Example of AWS Virtual Machine Provising.

```
### AWS dependency configuration
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">=5.0.0"
    }
  }
  required_version = ">=1.0.0"
}

### Creation of VM instance on AWS infra
resource "aws_instance" "example" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  tags = {
    Name = "ExampleInstance"
  }
}
```

organized into units called *working directories*. Large-scale infrastructure projects often use multiple working directories, each with provider dependencies for better visibility and cohesion of the IaC code [1]. Listing 1 shows an HCL snippet for virtual machine provision. The configuration begins by specifying the required Terraform version, followed by the source of the AWS provider (e.g., "*hashicorp/aws*") and a dependency constraint (e.g., "$>= 5.0.0$") to ensure the compatibility of the provider version. Constraining provider versions is a recommended best practice[4] to improve reproducibility and avoid unexpected issues from automatic version upgrades. After configuring the provider, the developer defines the VM instance, including its image, type, and tags.

B. Semantic Versioning for Dependency Management

When specifying a version of Terraform (TF) providers, developers typically follow the *Semantic Versioning* (SemVer) convention (e.g., *3.0.0*, *2.0.0*) [13]. *SemVer*[5] is a widely adopted versioning scheme that enables maintainers to communicate the nature of changes introduced in each release, particularly their impact on backward compatibility. It uses a *Major.Minor.Patch* format, where: (i) *Major* version changes may introduce breaking changes; (ii) *Minor* versions add new features in a backward-compatible manner; and (iii) *Patch* versions include backward-compatible bug or security fixes. Moreover, SemVer supports *pre-release versions*, marked with a hyphen and label (i.e., *3.0.0-alpha.1*, *2.1.0-rc.2*). These versions are typically instable and are used for testing and early access. In the TF ecosystem, these pre-releases are not installed by default unless the TF developers explicitly use them, as they represent instability and are subject to breaking changes [13].

C. Dependency Management for Providers in IaC Code

The convention of SemVer allows TF users to make informed decisions when selecting TF provider versions that align with their infrastructure requirements. This structure not only facilitates the manual selection of versions, but also governs automated update behaviors through *dependency constraints*. A dependency constraint typically consists of an *operator* and a *version number*.

Operators restrict allowable versions. For example, $= 2.0.0$ pins the provider to an exact version, often used for critical features or security fixes. Range-based operators (e.g., $>= 2.0.0$) allow automatic updates within a defined range. TF also supports exclusion ($!=$), comparison ($>$, $<$, $>=$, $<=$), and pessimistic constraints ($\sim>$), which limit upgrades to the rightmost number (e.g., $\sim> 2.0.0$ allows updates up to but not including *2.1.0*). In the case where TF developers do not intentionally specify the dependency constraint (i.e., no operators used and no version specified, latest), TF core will automatically install the latest version of the provider dependency, according to official documentation [13].

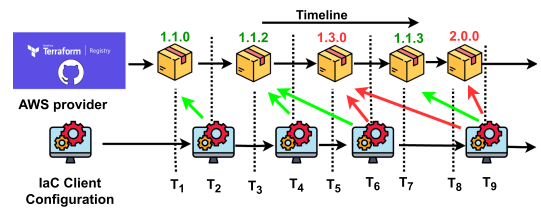


Fig. 1. Technical lag scenarios in Terraform code.

D. Technical Lag in IaC Configuration

Technical lag refers to the delay between the availability of a newer dependency version and its adoption in the client code [14], [15]. In the IaC context, technical lag often arises when practitioners restrict dependency versions in ways that limit automatic updates, despite frequent releases of provider artifacts [10]. For example, consider a developer who specifies the constraint *version* = "$\sim> 1.1.0$", which allows only patch-level updates beyond version *1.1.0*. As shown in Fig. 1, at time T_2, TF installs version *1.1.0*, the most recent version that satisfies the constraint. Over time, TF can automatically upgrade to *1.1.2* (T_4) and later *1.1.3* (T_9), but excludes older minor or major versions such as *1.3.0* and *2.0.0*. As a result, the configuration remains outdated and cannot benefit from the added features or improvements of the upstream provider.

E. Motivating Example: AWS Terraform Provider Updates

To illustrate the real-world consequences of technical lag and to motivate the need for this study, consider the example of the AWS Terraform provider. This provider released the version `4.0.0` in February 2022, introducing substantial changes, expanding support to more than 897 managed cloud resources, and offering enhancements in performance, security, and compatibility [10].

This breaking change significantly affected the *aws_s3_bucket* resource, which is widely used for various purposes, including storing backups, hosting static websites, serving media files, and supporting Big Data analytics. A

[4]https://developer.hashicorp.com/terraform/language/providers
[5]https://semver.org/

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

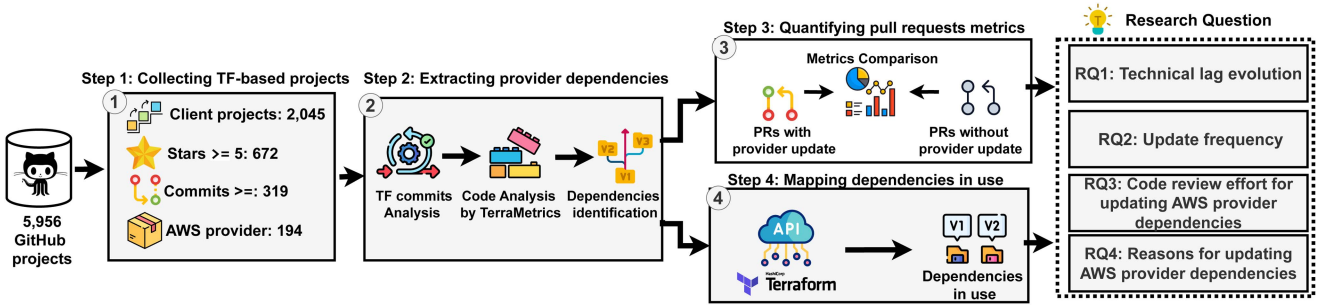


Fig. 2. An overview of our study approach.

key aspect of this change was its impact on the resource's configuration, particularly the version attribute [10], which initiated a broad discussion with more than 90 comments from over 40 developers. Due to the severity of this breaking change, TF provided special documentation to help practitioners carefully address the issues and migrate to newer resources [10].

Due to the widespread usage of S3 buckets for AWS data storage, TF practitioners may be reluctant to upgrade to newer versions (e.g., *4.0.0*) to avoid possible service interruptions and extensive troubleshooting. For example, in the GitHub project `zenml-io/mlstacks`,[6] which uses TF files to configure the infrastructure for machine learning workflows, a committer specified the provider version dependency as $>= 3.54.0, <4.0$ to circumvent the installation of version *4.0.0*. This indicates that TF practitioners can restrict provider versions to prevent major releases such as *4.0.0*, which bring breaking changes that could cause infrastructure outages. However, deliberately avoiding newer versions can cause dependencies to be outdated behind upstream releases. Over time, this delay in adopting updates may accumulate, especially when developers do not explicitly revise dependency constraints. As a result, maintaining outdated dependencies can increase long-term maintenance costs and introduce operational risks (i.e., security risks, breaking changes). These practical challenges highlight the importance of understanding how TF practitioners manage their provider dependencies. This motivates our investigation into the technical lag, prevalence of dependency updates, the associated effort, and the decision-making processes involved in updating the AWS provider dependencies.

III. Empirical Study Design

The *goal* of this empirical study is to investigate infrastructure developer practices to update their AWS provider dependencies in their IaC client configuration files. The remainder of this section details the characteristics of the context and the settings of the empirical study for data collection, as illustrated in Fig. 2.

A. Step 1: Collecting TF-Based Projects

The context of this study is the complete code change history of Terraform-based open-source projects hosted on GitHub. We focus on Terraform due to its widespread adoption and the increasing popularity of its configuration language, HCL. According to the GitHub 2024 Octoverse Report,[7] HCL experienced a 25% year-over-year growth in usage between 2023 and 2024, highlighting its growing relevance in modern infrastructure management [5]. The next section describes our data collection process.

Data Collection: We use the GitHub Code Search API[8] to collect software projects containing at least one Terraform file (i.e., files with "*.tf*" extension), following previous studies [16], [17], [18]. Our search query includes "*extension:tf sort:stars order:desc*" to identify the TF files indexed by the GitHub API and prioritize popular projects. Each search returns up to 1,000 indexed files, which may vary over time [16]. To ensure coverage, we repeated the query on multiple days between October 2024 and January 2025, resulting in 5,956 unique projects.

Data Filtering: Prior work on mining software repositories [19], [20], [21] emphasized the need for careful preprocessing to avoid including irrelevant projects, which can lead to conclusion instability over time. To address this, in the following, we apply a set of filters to refine our initial TF-based projects and retain only relevant client projects.

Filter #1. Select client TF-based projects: As stated in the TF documentation [22], the community can share TF files and providers as GitHub projects, making reusable components available for client projects. These repositories function as owner systems in real-world applications, specifically codified to support clients within the TF ecosystem. As our goal is to analyze the update of dependencies in client configuration, keeping the owner system can bias our conclusions. To exclude these, we first gather the full list of modules and providers from the TF Registry [23] and their source locations on GitHub. We then remove them, reducing the number of projects from 5,956 to 5,803. Upon analyzing the characteristics of the 153 excluded repositories, we found that they use HCL as the primary language or include the keyword `terraform` in their repository names. Based on that, we exclude the projects with HCL as primary language or whose names contain a `terraform` keyword in the full name of the repository. These two filters reduced the previous project list from 5,803 to 2,045 projects.

[6]https://tinyurl.com/4phu2utz

[7]The GitHub 2024 Octoverse Report provides insights into trends in software development across GitHub repositories, including language usage statistics, contribution activity, and community engagement.

[8]https://docs.github.com/en/rest?apiVersion=2022-11-28

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

TABLE I
STATISTICS OF THE 194 SELECTED TF-BASED PROJECTS

Metric	Min.	Median ($\bar{x}$)	Mean (μ)	Std Dev (σ)	Max.
Age (months)	6.96	60.28	67.71	32.98	174.54
Maintainers	3	41.5	125.37	222.75	1,283
Commits	102	1,723.5	6,078.19	12,003.31	102,267
Stars	5	198	1,998.9	4,718.38	30,426

TABLE II
STATISTICS ON THE SELECTED PROVIDERS

Provider	# Projects	# Commits	# Working directories
AWS	194	22,683	1,773
Google	61	6,667	270
Azure	36	3,599	122
Kubernetes	36	3,526	248

Filter #2. Select active projects: First, we remove fork repositories (i.e., projects that are copies of other repositories), reducing our dataset from 2,045 to 2,039. Then, we exclude archived projects (i.e., projects marked as read-only and no longer actively maintained), bringing the dataset down to 1,975 projects. We then select projects with a minimum of 5 stars, indicating minimal community interest. This filter reduces our dataset from 1,975 to 672 projects. Since our objective is to analyze the dependencies of AWS providers, we examine the commits of 672 projects to identify those using the `provider "aws"` statement. This filtering process results in a dataset of 319 projects that use AWS as a provider dependency. To ensure diverse contributors, we remove those having fewer than 3 contributors, yielding 268 projects. We then remove projects with fewer than 100 commits to ensure substantial development activity. As a result, we reduce our context from 268 to 242 projects. Furthermore, considering that the initial release (v0.1.0) of the AWS provider occurred on June 20, 2017, as documented in the release notes,[9] we limit our analysis to projects that include commits modifying Terraform (TF) files from this date up to the current year (2025). This filtering step excludes 4 projects, yielding a final dataset of 238 projects.

Filter #3. Select production-ready/mature projects: In this step, we eliminate the toy or prototype projects created primarily to learn IaC techniques (e.g., *"sofwerx/swx-devops"*, *"100daysofdevops/100daysofdevops"*) or using TF files as test artifacts (*"SonarSource/sonar-iac"* and *"semgrep/semgrep"*) instead of deployment scripts. We manually reviewed the 238 selected projects to filter out non-production projects, analyzing their names, descriptions, and documentation files. After this evaluation, we retained 194 projects that met the maturity and real-world applicability criteria.

Table I summarizes the characteristics of the 194 selected projects, including stars, contributors, commits, and age. Commit counts range from 102 to over 102 K, and project age spans from five months to 14 years, reflecting a diverse mix of new and mature projects. A detailed analysis is provided in our replication package [11].

B. Step 2: Extracting Provider Dependencies

This section outlines our two-step approach for analyzing the evolution of TF provider dependencies: (i) extracting historical dependency states, and (ii) identifying version updates over time to characterize update behavior at commit level.

Retrieving the history of provider dependencies: We collect historical data on provider dependency changes by analyzing commits that modify TF ("`*.tf`") files across 194 selected open-source projects. By tracking these changes, we aim to identify when a developer upgrades or downgrades a dependency at a given point in time. To extract commit-level changes, we use PyDriller,[10] a Python framework for mining Git repositories, to iterate over 23,404 commits that modify at least one "`*.tf`" file in the 194 projects. For each commit, we check out the repository (i.e., applying "*git checkout*" command) and extract only the TF files from their respective working directories, capturing the state of the IaC configurations at that point in time. This process analyzes 2,271 working directories in all commits. We use Terrametrics [24], an open-source static analysis tool for Terraform, to construct an Abstract Syntax Tree (AST) for each "`*.tf`" file. We then traverse the AST nodes to identify the provider dependencies in each working directory. During AST node's traversal, we extract each provider's *source* (Terraform Registry location) and *version* (dependency constraint), then parse the version constraint to obtain the operator and version number(s).

In modern cloud infrastructure, providers such as AWS, Google Cloud Platform (GCP), and Microsoft Azure often co-use multi-cloud deployments to optimize costs and improve sustainability [1], [25]. Since AWS is used in all 194 projects in our dataset, we use it as the reference point for further multi-provider analysis of the update practices. We then identify the three most frequently co-used TF providers with AWS, including: Google (used in 61 projects), Azure (36 projects), and Kubernetes (36 projects), as shown in Table II. Although Kubernetes is not a cloud provider, we include it due to its frequent provisioning and management via TF using the Kubernetes provider. Table II shows that the selected providers have enough commits and directory representation to analyze update behaviors among different TF providers.

Identifying updates of provider dependencies: Tracking provider dependency updates (i.e., changes in version constraints or operators) is challenging due to Git's branching and parallel development [26]. Following He et al. [27], we construct a directed acyclic graph (DAG) per project to compare provider dependencies between parent and child commits across branches. A dependency update is defined as a change in the constraint of an existing provider between commits c_1 (parent) and c_2 (child) in a given working directory w. Let P_1 and P_2 be the sets of provider dependencies in w at c_1 and c_2, respectively. Each dependency is a tuple of source and constraint, and version changes are captured as: $V_{c_2} = \{pro_i \mid pro_i \in P_1 \wedge pro_i \in P_2 \wedge \mathrm{ver}(pro_i, c_1) \neq \mathrm{ver}(pro_i, c_2)\}$. It is

[9]https://tinyurl.com/4x9d4fuv

[10]https://github.com/ishepard/pydriller

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

TABLE III
MONTHLY AVERAGE OF VERSIONING ACTIVITY

Provider	# Downloads	# Releases	# Major	# Minor	# Patch
AWS	4B	421 (4.58)	13 (0.14)	360 (3.91)	53 (0.58)
Google	1.2B	339 (3.77)	17 (0.19)	281 (3.12)	45 (0.50)
Azure	904.6M	336 (3.65)	5 (0.05)	283 (3.08)	47 (0.51)
Kubernetes	697.5M	86 (1.48)	3 (0.05)	49 (0.84)	34 (0.59)

Values are in the form of $Total(MonthlyAverage)$ where $Total$ is the total count of versions per provider from 2017 to 2025, and $MonthlyAverage$ is the mean of versions per month.

worthy to mention that in our study, we do not consider the analysis of pre-releases, similar to prior work [14], [28], as they often represent instable versions subject to breaking changes and can misinterpret version ordering.

C. Step 3: Quantifying Pull Requests Metrics

To assess the code review effort for provider dependency updates, we analyze pull requests (PRs) associated with commits that update these dependencies. PRs are the primary mechanism through which code changes are reviewed, discussed, and approved in collaborative development workflows [29]. They encapsulate both social and technical aspects of the review process, making them a well-established proxy for assessing the impact of provider dependency updates on review complexity and effort [30], [31]. In this step, we focus on the AWS provider, which is the most widely used provider in our dataset. We identify 390 commits that update the AWS provider constraints and extract 369 of their respective merged PRs across 61 TF projects. Not all updates proceed through PRs, as some commits bypass review. For further comparison of PRs with and without AWS provider updates, we randomly collect a set of 369 merged PRs unrelated to AWS provider updates as a control group from a total of 253,900. Meanwhile, we compute a set of code review metrics and PR size metrics for each PR from the two groups, following previous work [32], [33], [34], [35]. These metrics include: (i) *number of reviewers*, (ii) *number of review comments*, (iii) *number of inline comments*, (iv) *review duration (hours)*, (v) *number of changed subsystems*, (vi) *number of changed directories*, (vii) *number of commits*, (viii) *code churn*, and (ix) *number of changed files*.

D. Step 4: Mapping Dependencies in Use

To complement our commit-level analysis and assess how developers adopt available provider updates, we collect the official release dates of these dependencies. For each provider dependency (AWS, Google, Azure, Kubernetes), we query its source repository to retrieve metadata, including release dates and corresponding versions, which follow the semantic versioning scheme (i.e., major.minor.patch). To systematically retrieve this information, we leverage the TF Registry API [22] via a `GET`,[11] where $source_name$ corresponds to the source identifier declared in the provider block [22].

Table III reports their average monthly release activity. AWS exhibits the highest release frequency, averaging ∼4 releases/month, followed by Google and Azure with similar rhythms. Kubernetes shows a slower pace (∼1 release/month). In Table III, minor versions dominate for AWS, Google, and Azure, averaging ∼3/month, suggesting a focus on incremental improvements with backward compatibility from their maintainers.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

This section presents and discusses the experimental results for our four research questions.

A. (RQ1) How long is the technical lag in AWS provider dependencies in Terraform-based projects?

Motivation: Prior studies on traditional dependency updates [14], [36], [37], [38] show that outdated dependencies can lead to security risks and incompatibilities, an issue known as technical lag (see Section II). Although TF is a popular IaC tool, little is known about the persistence of outdated AWS provider dependencies in TF codebases. In modern cloud deployments, it is common for TF projects to rely on multiple cloud providers simultaneously, such as AWS, Google Cloud, Azure, and Kubernetes, to meet diverse infrastructure needs. This multi-provider usage introduces complexity in dependency management and may influence the frequency or priority of updating a specific provider, like AWS. To investigate this, RQ1 examines technical lag in 194 TF projects that include the AWS provider, focusing on how long AWS provider dependencies remain outdated, particularly in environments where AWS is used alongside other providers (e.g., Google Cloud, Azure, and Kubernetes).

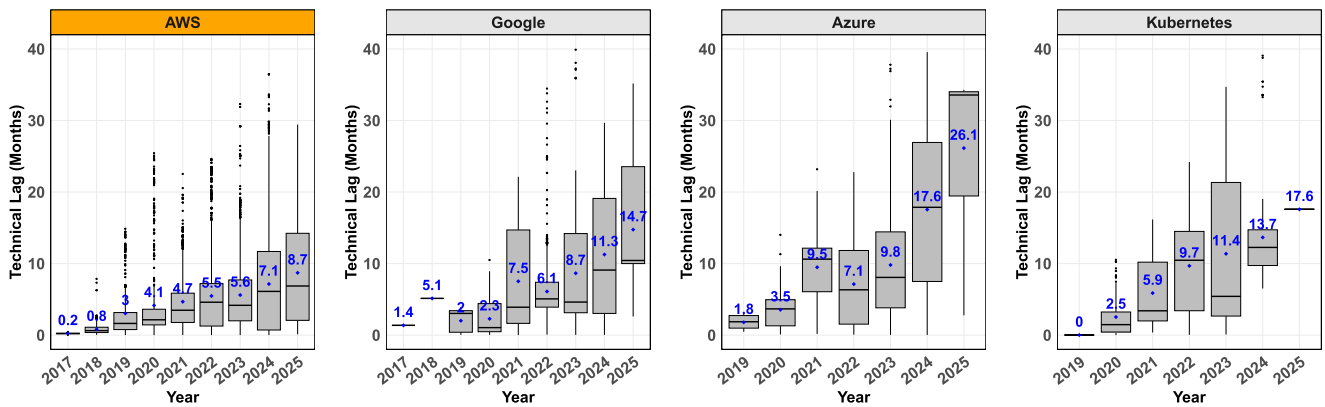
Approach: To quantify technical lag in terms of time for AWS, Google, Azure, and Kubernetes provider dependencies, we adapt the approach outlined in previous studies [14], [15]. Specifically, for each provider dependency d within a working directory w at a given commit date t, we compute the technical lag as the time elapsed $\Delta(d, t)$ since the earliest uninstalled release that is newer than the version currently used at time t.

We refer to the set of such releases as $missed(d, t)$, which is the set of all newer versions of a dependency d that were already available at time t but were not installed due to the specified dependency constraints (i.e., operator-based filtering). To determine this set, we extract the version constraint expressions (e.g., $\sim>$ 2.1.0, $>=$ 3.0.0) from the TF files and resolve the version that would have been installed by the TF core at time t based on the official version selection rules.[12] By default, the TF core selects the latest available version that satisfies the specified dependency constraint. We then compare this resolved version (i.e., a version actually in use at time t) with the set of all *upstream* releases available at that time in the TF Registry. Any release that is higher than the resolved version but does not satisfy the constraint is considered part of $missed(d, t)$, following the same method used by Decan et al. [14]. This allows us to calculate the technical lag $\Delta(d, t)$ as the difference

[11] https://registry.terraform.io/v1/providers/source_name/versions

[12] https://tinyurl.com/5d4antb3

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.



(a) AWS Time-lag. (b) Google Time-lag. (c) Azure Time-lag. (d) Kubernetes Time-lag.

Fig. 3. Evolution of technical lag across the AWS, Google, Azure, and Kubernetes provider's dependencies in the 194 Terraform-based projects.

between t and the release date of the missing earliest version, as formalized in Equation 1.

To account for the fact that TF practitioners often organize their infrastructure into multiple working directories [1], we compute technical lag by averaging those directories that include at least one provider dependency. This approach reduces the influence of infrequently updated provider dependencies, resulting in more stable and representative lag estimates between projects with diverse characteristics (i.e., see Table I). Based on this refinement, we calculate the overall technical lag for a provider pr_i at time t by aggregating the individual lags across all working directories $W_{pr_i}(t)$ where the provider is used. We then compute the average by dividing the total lag by the number of such working directories, as shown in Equation 2.

$$\Delta(d,t) = t - \min\{r_{\text{date}} \mid r \in missed(d,t)\} \quad (1)$$

$$\Delta_{\text{norm}}(pr_i,t) = \frac{1}{|W_{pr_i}(t)|} \sum_{d \in W_{pr_i}(t)} \Delta(d,t) \quad (2)$$

Results: Fig. 3 illustrates the yearly evolution of the technical lag exhibited by AWS provider dependencies, along with the co-used providers: Google, Azure, and Kubernetes.

Observation #1. The technical lag associated with the AWS provider shows a continuous upward trend, reaching an average of approximately 9 months in 2025: As shown in Fig. 3, the technical lag was relatively low between 2017 and 2019, ranging from an average of 6 days to 3 months. This finding indicates that client projects tend to be initially aligned more with recent AWS provider releases, possibly due to the freshness and active adoption of the provider following its initial release in 2017. Beginning in 2019, the technical lag exhibited a steady increase from an average of 3 months to 5.5 months by 2022. Looking more closely at this, we observe that this period experienced significant releases such as v3.0.0 (July 2020),[13] which introduced four major enhancements: (*i*) updates to the Amazon Certificate Manager (ACM) resources, including changes to how certificate attributes are handled in the state; (*ii*) the removal of hashing from state storage to simplify state management and improve security; (*iii*) improved authentication ordering, allowing more predictable and flexible provider initialization; and (*iv*) the formal deprecation of Terraform Core version 0.11, encouraging users to upgrade to newer versions for continued compatibility and support. Consequently, such major changes may have led to delays in adoption due to the human effort and maintenance costs required to update existing configurations. In 2022, further increases in technical lag can be attributed to ecosystem-level factors, particularly the breaking changes introduced in v4.0.0 (February 2022).[14] Such major version changes often require substantial updates to client configurations, which can lead to delayed adoption. Starting from 2023, the average technical lag continued to rise, increasing by approximately 27% in 2024 and showing further growth in 2025. This upward trend suggests a growing difficulty or reluctance among developers to keep AWS provider dependencies up-to-date.

Observation #2. Google, Azure, and Kubernetes providers exhibited a consistent upward trend in technical lag from 2022 to 2025: As shown in Fig. 3, the technical lag of Google, Azure, and Kubernetes provider dependencies, when used alongside AWS providers in multi-cloud environments, follows an overall increasing trend. In 2017, these providers exhibited relatively low technical lag, ranging from 0 to 1.8 months on average. This aligns with AWS provider dependencies, where newly released provider versions tend to have lower lag during their initial adoption phase. Between 2018 and 2022, Google and Azure providers exhibited an instable pattern of technical lag. For example, both providers experienced a notable increase in lag in 2021, followed by a decrease in 2022. This fluctuation can be linked to several disruptive technical events introduced by the maintainers during that period. For example, in 2021, the changes in the Google TF provider were introduced to fix 11 bugs that could affect widely used resources [15] (e.g., containers, domain name system). Similarly, the TF Azure provider introduced significant updates between 2021 and 2022, including deprecations and breaking

[13] https://tinyurl.com/wdp44wf9, https://tinyurl.com/t86pb7rn

[14] https://tinyurl.com/52xvbwum

[15] https://tinyurl.com/35tv3czb

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

changes in across different versions from v2.88.0[16] to the major v3.0.0 release.[17] These updates likely caused delays in adoption due to the migration effort involved. From 2022 through early 2025, technical lag increased significantly for the Google, Azure, and Kubernetes providers. In particular, by 2024, Azure provider dependencies reached an average technical lag of approximately 17 months, while Kubernetes provider dependencies exhibited an average lag of around 13 months.

B. (RQ2) How frequently are AWS provider dependencies updated in Terraform-based projects ?

Motivation: Previous work [1], [8], [9] has shown that infrastructure dependencies evolve rapidly due to bug fixes, security patches, performance gains, and new features. Thus, TF maintainers frequently release their providers, such as AWS, Google, and Azure (about three versions per week). Therefore, this RQ2 quantifies how often real-world projects adopt these updates.

Approach: To address this RQ, we analyze our collected updates for the four studied providers (AWS, Google, Azure, and Kubernetes), as described in Section III. We compute the update frequency by normalizing the unique commits that update a given provider pro_i in project p_j, divided by the total number of unique commits that declare or use that provider, as described in Equation 3. We report the number of updated projects (i.e., updated / declared), updated working directories (i.e., directories where the provider is updated / total where it is used), and the proportion of commits involving operator changes, upgrades, or downgrades. Updates are also categorized by type: major, minor, or patch.

$$\text{Update}(pro_i, p_j) = \left(\frac{\sum_{c \in C} \{1, c \text{ updates } pro_i \text{ in } p_j\}}{|\{c \in C \mid \text{uses}(c, pro_i, p_j)\}|} \right) \times \% \tag{3}$$

Operators in dependency constraints significantly influence the evolution of technical lag. Restrictive operators (e.g., $=$, $<$) limit the adoption of the version and can lead to outdated dependencies. In contrast, permissive operators (e.g., $>=$, or use of empty declaration for latest downloads) support flexibility, but can introduce instability if not regularly reviewed. To investigate their impact, we analyze (i) operator type changes in commits that explicitly modify operators, and (ii) operator usage across working directory declarations. The former reveals the intent of developers and the evolving versioning strategies, while the latter reflects practical constraints during infrastructure provisioning. Together, these perspectives provide a comprehensive understanding of how operator practices impact technical lag in TF-based projects.

Results: Fig. 4 presents boxplots illustrating the percentage of commits that include upgrades and downgrades of provider's dependencies (i.e., AWS, Google, Azure, Kubernetes). Additionally, Table V provides statistics on the number of projects that perform provider dependency updates and the corresponding number of updated working directories.

TABLE IV
DISTRIBUTION OF DEPENDENCY CONSTRAINT OPERATORS PER PROVIDER. EACH PROVIDER IS SHOWN WITH TWO ROWS: ONE FOR COMMITS THAT *INTRODUCED* AN OPERATOR (**INTRODUCED**), AND ONE FOR WORKING DIRECTORIES THAT *USED* IT (**USED IN**)

Provider	Type	~>	=	>=	<=	<	>	!=	latest	Total
AWS	Introduced	33.6%	22.8%	18.8%	0.7%	2.7%	0.7%	0.7%	20.1%	149
	Used in	25.06%	12.7%	8.86%	0.19%	0.44%	0.05%	0.83%	51.87%	2,055
Google	Introduced	35.3%	20.6%	11.8%	5.9%	2.9%	0.0%	0.0%	23.5%	34
	Used in	28.17%	14.86%	13.0%	6.19%	0.62%	0.0%	0.0%	37.15%	323
Azure	Introduced	56.2%	18.8%	6.2%	6.2%	0.0%	0.0%	0.0%	12.5%	16
	Used in	22.38%	40.56%	6.99%	1.4%	1.4%	0.0%	0.0%	27.27%	143
Kubernetes	Introduced	33.3%	50.0%	16.7%	0.0%	0.0%	0.0%	0.0%	0.0%	6
	Used in	32.02%	16.6%	11.86%	0.4%	0.0%	0.0%	0.0%	39.13%	253

Table header shows provider, change type, and TF operators. The total column summarizes introduced operators in commits changing operators, and use in directories.
Totals may differ from earlier Table 5, which count unique commits. current table includes all dependency changes, including those with multiple operators or multiple updates per commit.

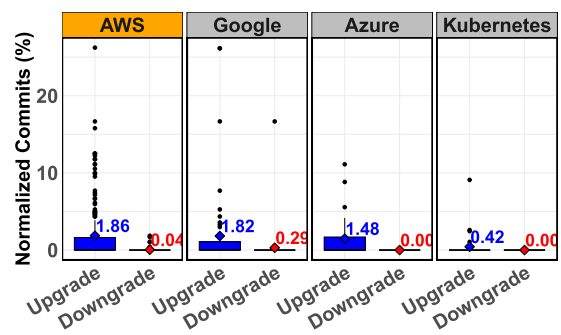


Fig. 4. Box plots of normalized upgrade and downgrade commits for AWS, Google, Azure, and Kubernetes providers across 194 TF projects.

Observation #3. Terraform developers rarely update AWS and Google providers; only 20% of working directories receive updates, and the average frequency of upgrade commits is approximately 1.82-1.86%: From Fig. 4, we observe that, on average, around 1.82 to 1.86% of the commits that use at least one AWS or Google provider dependency perform an upgrade. In these upgrades, 11% of the 194 projects exhibit an upgrade frequency greater than 10%. For example, in *grafana/loki*,[18] a log-aggregation system, the AWS provider shows an upgrade frequency of approximately 26% in the commits that declare its usage. In particular, on this project, we find that bots or automated dependency management tools typically perform these updates (e.g., #ea1db2,[19] #6e5191[20]). According to Table V, 414 commits are responsible for upgrading AWS provider dependencies, where 1.9% of the commits consist of major upgrades, 56.8% are for minor upgrades, and 42.3% are for patch upgrades. A similar pattern is observed for Google provider dependecies, where 58% of upgrades concern minor upgrades. This could indicate that the prevalence of minor updates may indicate that TF developers aim to remain aligned with frequent provider

[16]https://tinyurl.com/bdem52vx
[17]https://tinyurl.com/pw2zry99
[18]https://github.com/grafana/loki
[19]https://tinyurl.com/y5hzzsyw
[20]https://tinyurl.com/54spzck6

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

TABLE V
STATISTICS ON COMMIT CHANGES FOR AWS, GOOGLE, AZURE, AND KUBERNETES PROVIDERS

Provider Type	# Updated Projects	# Updated Working Dires.	# Operator Changes	Version Changes ← downgrades	Version Changes upgrades →	Change Type
AWS	78/194 (40%)	355/1,773 (20%)	139/573 (24.3%)	20/573 (3.5%)	414/573 (72.2%)	Total Updates (#)
				10%	1.9%	Major Updates (%)
				65%	56.8%	Minor Updates (%)
				25%	42.3%	Patch Updates (%)
Google	22/61 (36%)	53/270 (20%)	29/120 (24.2%)	5/120 (4.2%)	86/120 (71.7%)	Total Updates (#)
				0%	3.5%	Major Updates (%)
				40%	58.1%	Minor Updates (%)
				60%	39.5%	Patch Updates (%)
Azure	18/36 (50%)	35/122 (29%)	16/55 (29.1%)	0/55 (0%)	39/55 (70.9%)	Total Updates (#)
				0%	0%	Major Updates (%)
				0%	56.4%	Minor Updates (%)
				0%	43.6%	Patch Updates (%)
Kubernetes	4/36 (11%)	65/248 (26%)	6/21 (28.6%)	0/21 (0%)	15/21 (71.4%)	Total Updates (#)
				0%	6.7%	Major Updates (%)
				0%	86.7%	Minor Updates (%)
				0%	6.7%	Patch Updates (%)

releases, as also reflected in Table III. From the semver definition [39], this could also suggest that TF developers receive new functionalities and features in a backward-compatible manner. In contrast, downgrades of AWS and Google provider dependencies are rare, occurring on average around 0.03% and 0.42% of TF commits, respectively. This suggests that developers prefer upgrading provider versions rather than reverting to older ones.

Observation #4. Updates to Azure providers are propagated across a larger share of working directories (29%) than AWS providers updates: From Table V, we observe that the proportion of updated working directories is 29% for Azure and 26% for Kubernetes providers. However, as shown in Fig. 4, the actual upgrade frequency for Azure and Kubernetes is relatively low, only 1.48% and 0.42% of commits, respectively, compared to AWS and Google. This discrepancy could be related to the way TF developers use the providers. Since most selected projects are primarily designed for AWS provisioning, they are expected to include many working directories to configure diverse infrastructure components. As a result, updating AWS dependencies may be more labor-intensive than Azure, which can be used in more focused environments or specialized parts of the infrastructure.

Observation #5. Across all providers, operator updates account for 24% to 29% of version changes, indicating that TF developers adjust not only version numbers but also the constraints on allowable versions: As shown in Table V, updates to dependency constraints involving operator changes represent 24.3%, 24.2%, 29.1%, and 28.6% of version changes for the AWS, Google, Azure, and Kubernetes providers, respectively. This suggests that when TF developers update provider dependencies, they modify not just the version itself (i.e., whether it is an upgrade or a downgrade) but also the associated version constraint operators, which directly affect how versions are resolved. For instance, one developer highlights the importance of correctly using version constraint operators to avoid unintended behavior[21]: 💬 "*[...] In this commit, I've made use of the ~> version specifier, which means that the last provided version specifier is a lower bound. So, ~> 3.0 means ≥ 3.0, < 4.0 and ~> 0.7.2 means ≥ 0.7.2, < 0.8. I see this as a good trade off between pinning tightly and not pinning at all. Like this we can avoid bugs, rely on up-to-date documentation, without checking when something is introduced, and avoid breaking changes [...].*"

Observation #6. With Azure dependencies, TF developers tend to use more exact versions, which could increase technical lag compared to other providers' dependencies: Table IV presents the ratio of dependency constraint operators across the four analyzed providers' dependencies. For each provider, we report the percentage of commits that introduced specific operators and the percentage of working directories in which these operators were used. The analysis includes both restrictive operators (e.g., $=$, $<=$) and permissive ones (e.g., $>=$, $latest$). From Table IV, we observe that Azure exhibits a significantly higher dependence on equality constraints (40.56%), which can hinder the adoption of newer versions and contribute to a longer technical lag, as observed in RQ1. This behavior may reflect concerns about version stability or the complexity of migrating to newer releases, leading TF practitioners to intentionally specify versions. In particular, Azure dependencies exhibit the highest degree of technical lag during the study period (2022–2025). In contrast, we observed that AWS provider dependencies show a comparatively slower evolution in technical lag from 2022 to 2025, relative to the dependencies for Azure, Google, and Kubernetes. This trend may be partly due to the lower usage of restrictive operators, such as equality ($=$), which appears in only 12.7% of AWS working directories.

Observation #7. TF developers tend to introduce more rightmost operators to benefit from the latest fixes for lower migration costs: According to Table IV, AWS employs a more balanced versioning strategy compared to other providers. Specifically, more than half of the working directories that declare AWS dependencies (51.87%) use permissive version constraints, allowing them to adopt the latest available version of the dependencies. The remaining usage is distributed among the more restrictive

[21] https://tinyurl.com/265e94dz

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

operators, such as $\sim>$ (25.06%) and $=$ (12.7%), reflecting the project's selective constraints. This hybrid practice suggests that AWS practitioners often strive to balance staying up-to-date with the need to control update risks. As shown in Table IV, the $\sim>$ operator, which restricts updates to minor and patch versions, accounts for at least 33% of operator changes, a higher proportion than any other operator. This pattern suggests that TF practitioners tend to prioritize avoiding breaking changes while still benefiting from bug fixes and incremental improvements. However, this strategy may unintentionally contribute to technical lag, particularly if version constraints are not regularly reviewed and updated. For instance, in the *2i2c-org/infrastructure*[22] project, developers initially introduced the constraint $\sim> 3.0$ for a specific provider dependency (#81e5e[23]). In subsequent commits, this constraint remained unchanged, resulting in an increase in technical delay from 3 to 47 days. To eliminate the lag (technical lag $= 0$), the version constraint was updated later to $\sim> 4.15$ (#21f19[24]), allowing for the adoption of newer versions. This update occurred during a period when the provider maintained both the *3.x* and *4.x* release lines, issuing updates to the *3.x* series even after the *4.0.0* release. This practice, known as backporting, involves applying bug fixes and security patches to older major versions. Although backporting allows users to remain on older versions without missing critical fixes, it can also create ambiguity among TF developers about the urgency of migrating to newer major versions [40].

C. (RQ3) What is the code review effort in AWS dependency updates?

Motivation: Although previous studies on dependency management [15], [41] have focused on semantic versioning and update frequency, little is known about how developers review and experience AWS provider updates, despite their critical impact on infrastructure behavior. As shown in Table III, the AWS provider exhibits more frequent minor version releases compared to major or patch versions. These minor releases often introduce changes to resource definitions, API versions, or authentication models [42]. Although these are not classified as breaking changes by the versioning rules [39], such changes can still cause infrastructure misconfigurations or deployment failures in practice.[25] Furthermore, AWS technology is recognized for offering numerous options for larger enterprises [43], potentially expanding the scope of configuration and adding to operational complexity. This situation might increase the cognitive load of developers when performing code reviews and maintaining systems [1]. As a result, reviewing AWS provider updates and integrating them into client code can be a challenging task and requires specialized knowledge of Terraform and AWS services. Thus, this RQ3 analyzes how code review for TF related code changes involving AWS provider updates differs from those that do not include AWS provider updates. This helps to gain insight into the distinct challenges of AWS provider updates that merit further attention from researchers and tool developers to improve IaC dependency management.

Approach: We assess the review effort involved in updating the AWS provider dependencies by comparing pull requests (PRs) that include these updates with those that do not, using data from Section III-C. The treatment group consists of 369 PRs with AWS provider updates. To determine whether updating PRs requires significantly different review effort (e.g., changed files, code churn, review comments), we test the null hypothesis (H_0): *"There is no statistically significant difference in review effort metrics (e.g., number of changed files) between updating and non-updating AWS provider dependencies."* The alternative hypothesis (H_1) asserts a significant difference. Following prior work [44], [45], we remove outliers using the interquartile range (IQR) method [46], based on the number of changed files, since outliers may reflect refactoring (e.g., file renaming) rather than meaningful updates. Specifically, we exclude values below Q1-1.5× IQR or above Q3+1.5× IQR [45]. This removes 43 and 46 outliers from the updating and non-updating groups, respectively.

To evaluate H_0, we first assess the distribution of each metric using the *Shapiro–Wilk* test [47], which tests for normality, i.e., whether the mean, median, and mode are all equal and located at the center of a bell-shaped curve. We apply this test to the metrics of both the treatment and control groups. The results indicate that all distributions deviate from normality (p-value < 0.05). Therefore, we use non-parametric methods for comparison. Specifically, we apply *Mann–Whitney U* test [48] to compare the two independent groups (i.e., PRs with and without AWS provider updates) for each metric, which does not assume a normal distribution for comparison groups. To complement statistical significance testing, we calculate *Cliff's Delta* (δ), a non-parametric effect size measure that quantifies the magnitude of the observed difference between the groups. This is essential to assess whether statistically significant results also have practical relevance. We follow the interpretation thresholds suggested by Romano et al. [49]: negligible if $|\delta| < 0.147$, small for $0.147 \leq |\delta| < 0.33$, medium for $0.33 \leq |\delta| < 0.474$, and large for $|\delta| \geq 0.474$.

Results: Table VI shows the statistical comparison of the code review metrics of pull requests with/without updating the AWS provider dependencies.

Observation #8. Pull requests that update AWS provider dependencies differ significantly from non-updating ones regarding the number of changed files, their directories, and the number of commits: Results reveal that code review practices in pull requests involving changes in AWS provider dependencies differ significantly from those that do not update the AWS provider dependencies. From Table VI, we observe that the number of files changed in pull requests involving AWS provider updates has a median of 7 files compared to pull requests that do not update AWS providers having a median of 2 files. Furthermore, statistical tests show a significant difference in the number of changed files (i.e., *p-value* < 0.05) with a large effect size between the two groups. Moreover, Table VI shows that the number of directories changed in pull requests with

[22] https://github.com/2i2c-org/infrastructure
[23] https://tinyurl.com/4wus35z5
[24] https://tinyurl.com/3eh537fa
[25] https://spacelift.io/blog/terraform-version

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
STATISTICS OF PROVIDER DEPENDENCY ACTIVITY EFFORTS

Metric	*Updating AWS Providers*						*Non-updating AWS Providers*						*Statistical tests*	
	Min	Q1	Median	Mean	Q3	Max	Min	Q1	Median	Mean	Q3	Max	P-value	δ (Effect size)
Number of reviewers	0	1	1	1.27	2	5	0	1	1	1.19	1	5	0.3339	0.0381 (negligible)
Number of review comments	0	0	0	0.38	1	5	0	0	0	0.31	1	3	0.5965	0.0186 (negligible)
Number of inline comments	0	0	0	1.62	0	57	0	0	0	0.75	0	39	0.2837	0.0352 (negligible)
Review duration (Hours)	0	0.77	21.12	154.76	105.55	12,535.6	0	1.03	13.72	211.94	88.5	6,913.36	0.6362	0.0215 (negligible)
Number of changed subsystems	1	1	1	2.41	3	12	1	1	1	1.48	2	8	**0.0000**	0.2211 (small)
Number of changed directories	1	2	4	6.39	9	40	1	1	1	2.41	3	15	**0.0000**	0.4313 (medium)
Number of commits	1	1	2	5.85	8	58	1	1	2	3.67	4	156	**0.0000**	0.1849 (small)
Code churn	2	23.5	118	1,066.89	453.5	22,936	0	6	30	208.19	130	12,989	**0.0000**	0.3069 (small)
Number of changed files	1	3	7	10.24	16	45	1	1	2	3.51	4	16	**0.0000**	0.5066 (large)

the update of AWS providers (i.e., a median of 4 directories) is significantly different from those for non-updating any AWS provider dependency (i.e., a median of 1 directory), with a medium effect size. For the code churn metric, we observe that, although statistically significant, the small effect size for code churn suggests that while differences exist, they may not strongly impact the overall review effort. These findings align with previous work in production code [50], [51], [52], [53], which showed that larger, more scattered changes increase review difficulty.

These results reveal that AWS dependency changes could affect several files. For example, a developer performed a pull request PR#13135[26] in the *fleetdm/fleet* project with 7 changed files to update the Amazon Elastic Kubernetes Service (EKS) configurations (i.e., 💬 "*Add changes needed for EKS upgrade*"). In this example, the developer made two commits, where one of which (#0e9c2bf[27]) aimed to revert the configuration to fix a security vulnerability (i.e., 💬 "*Fix for sandbox vuln processing*"), address a bug (i.e., 💬 "*fix bug in deprovisioner*"), and optimize the behavior of resource provisioning (i.e., 💬 *Scale down and delete cronjob for expired instances*"). Making such updates to AWS provider dependencies can necessitate additional development efforts to ensure compatibility, which involve changing several files from different directories.

Furthermore, results show that the number of reviewers, review comments, and inline comments, do not show a notable difference (i.e., *p-value* ≥ 0.05), with comparable medians across these metrics. This could indicate that the update review practices for AWS provider dependencies are fundamentally similar to those of non-updating AWS providers commenting on pull request changes. Although pull requests updating the AWS provider show a higher median merge time of 21 hours than non-updating ones ($\approx$ 14 hours), the statistical test shows that this difference is not significant. This suggests the absence of a consistent pattern in merge durations between the two groups. These results indicate that, while some AWS provider updates may take longer to merge, likely due to their potential impact on infrastructure, this is not a uniformly observed trend across the PRs in both groups.

[26]https://github.com/fleetdm/fleet/pull/13135
[27]https://tinyurl.com/3ypk5d6n

D. (RQ4) What are the developer's motivations when updating their AWS provider dependencies?

Motivation: While prior research on traditional third-party library dependencies has explored motivations for updates, such as security and performance improvements [36], [54], [55], little is known about what drives updates to AWS provider dependencies in TF configurations. RQ1-RQ3 quantitatively investigated technical lag and code review features. However, they do not uncover the fundamental motivations that inform update decisions. To address this, RQ4 performs a qualitative analysis to identify key factors influencing these updates and construct a developer motivation taxonomy.

Approach: To address RQ4, which aims to uncover the motivations behind AWS provider dependency updates, we qualitatively analyze a set of 390 unique commits that update AWS provider dependency constraints. These commits are extracted from the 369 pull requests previously studied in RQ3 and represent all instances where the AWS provider dependency constraint was explicitly updated (e.g., a change from $\sim> 2.0.0$ to $>= 2.1.0$). In practice, a PR could encompass multiple commits that attempt to update AWS provider dependencies. Therefore, unlike RQ3, which focused on pull requests to examine the scope and review effort of updates, we focus here on commits as the unit of analysis to capture the fine-grained motivations behind individual version changes. To accurately infer the motivation behind each update, we examined both the code changes surrounding the AWS provider version block and the commit message, using them as the primary sources of context. When the commit message was insufficient or the change was part of a broader set of modifications, we additionally reviewed the title and body of the corresponding pull request to clarify the rationale behind the version update. However, motivation codes were only assigned when the available textual or structural evidence referred to the AWS provider change itself.

We adopt the *thematic analysis* method [56], a widely-used methodology in software engineering research to extract meaningful patterns from qualitative data [34], [35], [56], [57]. Thematic analysis involves six phases: (1) familiarization with the data, (2) generating initial codes, (3) searching for subthemes, (4) reviewing and refining subthemes, (5) defining overarching themes, and (6) producing the final narrative. Three authors with expertise in DevOps, IaC, and dependency management participated in the analysis. Their average respective experience

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

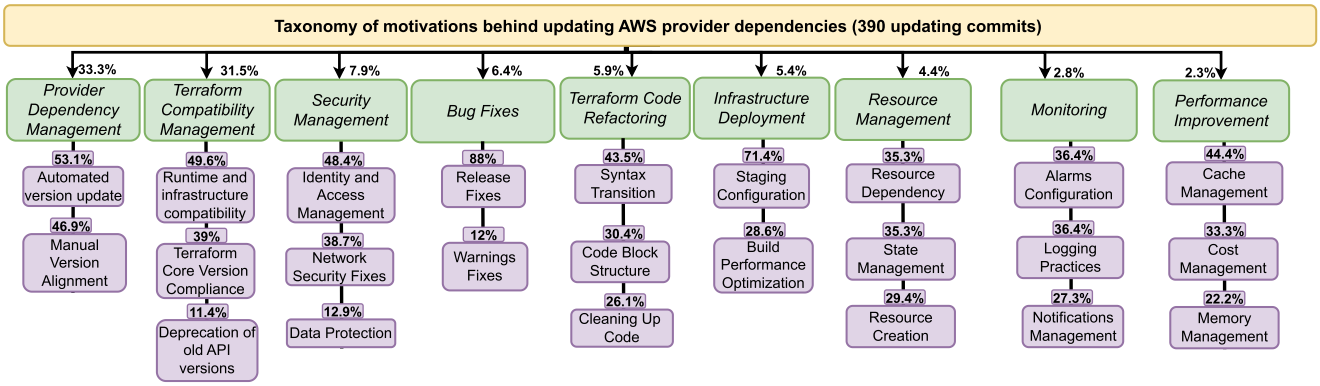


Fig. 5. Taxonomy of the main motivations for updating AWS provider dependencies.

spans four years in IaC, five years in DevOps, and over nine years in dependency management.

To ensure alignment in interpretation, two authors having at least four years of experience working on DevOps practices jointly reviewed an initial sample of 30 commits and their associated pull requests to calibrate the coding approach. Thereafter, they independently coded the entire dataset. The reliability between the authors was assessed using Cohen's kappa, yielding a value of $kappa = 0.69$, indicating substantial agreement [58]. The two authors then met to resolve the disagreements present in 22% of the coded items. Furthermore, the coders identified semantically similar but differently phrased labels in 4.3% of the dataset (e.g., "*fix broken releases*" vs. "*release fixes*"), which were merged during the reconciliation process. This led to a refined set of lower-level codes that were grouped into subthemes (e.g., "*syntax transition*" and "*TF code cleanup*" under the subtheme "*TF code refactoring*"), and eventually abstracted into higher-level themes. After further discussion, the two authors refined the coding to achieve complete consensus, resulting in a final agreement with a $kappa$ value of 1.0. To ensure both consistency in coding and alignment with domain-specific distinctions, a third author reviewed the final coding schema, examined ambiguous cases, and provided expert feedback. In particular, the third author contributed to refining thematic boundaries, such as clarifying the distinction between motivations related to Terraform Core and those concerning Terraform providers.

Results: Our thematic analysis identified nine motivations behind TF developers' updates to AWS provider dependencies: *Providers Dependency Management*, *Terraform Compatibility Management*, *Security Management*, *Bug Fixes*, *Terraform Code Refactoring*, *Infrastructure Deployment*, *Resource Management*, *Monitoring*, *Performance Improvement*. Fig. 5 presents the taxonomy of these motivations and their corresponding categories. We define each of the nine motivations in detail and provide representative examples for each in our Appendix A, included in our replication package.[28] These examples include direct links to relevant GitHub commits and pull requests, organized and indexed by category (e.g., [C1.1], [C2.3]).

[28]https://tinyurl.com/mwtk76v3

① *Providers Dependency Management:* This is the most prominent motivation, accounting for 33.3% of the updates. Developers frequently update their AWS provider dependencies to take advantage of newer features, stability improvements, and bug fixes introduced in recent versions. In this category, we found that 53% of included updates are automated provider version changes driven by dependency management tools such as dependent bots and automated solutions, which are configured to periodically check for and apply minor or patch upgrades (e.g., "💬 *Renovate Update Minor Updates*" [C1.1] Appendix A). In contrast, 46.9% of the updates in this category are manually performed to follow good maintenance practices and access new provider features in a controlled manner. (e.g., " 💬 *By pinning to the patch or minor versions, we may miss out on bugfixes and additions of relevance... Like this we can avoid bugs, rely on up-to-date documentation, without checking when something is introduced, and avoid breaking changes quite well.*" [C1.2] Appendix A).

② *Terraform Compatibility Management:* This category represents 31.5% of the total updates, where the main focus is maintaining the Terraform ecosystem's compatibility with the target cloud platforms. Approximately 49% of these updates address *Runtime and infrastructure compatibility* concerns related to ensuring compatibility between providers and runtime integration of such third-party modules or services (e.g., 💬 "*Update all our lambdas to Node20, the aws cli now supports this runtime, and 20 is currently LTS. A terraform upgrade also came along for the ride*" [C2.1] Appendix A). Out of these updates, 39% are related to *Terraform Core Version Compliance* to ensure the co-evolution of AWS provider dependency with the TF core version, often by pinning the minimum Terraform core version (e.g., 💬 "*this is step 1 of several steps to upgrade tf and aws providers associated with it.*" [C2.2] Appendix A). The remaining 11.4% involve provider updates to avoid deprecation of old API versions (e.g., "*terraform template_file is deprecated in 0.12. Use templatefile() instead. With a recent aws provider version, this has the added benefit of not redeploying the instance every time*" [C2.3] Appendix A).

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

③ *Security Management:* We found that 7.9% of the updates account for security-driven motivations. Of these, 48.4% turn around *Identity and Access Management*, where TF developers try to grant or fix access to AWS services (e.g., "*Add resource policy to allow access from corresponding orchestration account and update terraform version to allow for this*" [C3.1] Appendix A). Another 38.7% focus on *network security*, by adding private endpoints, sub-networks or configuring security groups (e.g., *"Networking TF: Upgraded Terraform 1.4.2 and fixed subnets for VPC endpoint interface"* [C3.2] in Appendix A). The remaining 12.9% of updates emphasize *data protection*, where developers enforce the encryption of the S3 buckets and data sensitive, and data storage (e.g., *"support for encrypting ssm params with AWS KMS keys"* [C3.3] Appendix A).

④ *Bug Fixes:* This category includes 6.4% of the total updates. Within this category, 88% of the updates are toward release fixes, which are typically related to fixing broken dependencies through downgrading providers version (e.g., "*Fix for broken AWS terraform provider >= 2.61.0... Roll back AWS terraform provider*" [C4.1] Appendix A), while the remaining 12% of bug fixes address warnings' fixes that do not troubleshooting the configuration execution in a point of time, but should be handled (e.g., "*silence further Terraform linter warnings*" [C4.2] Appendix A).

⑤ *Terraform Code Refactoring:* Refactoring-related motivations cover 5.9% of the updates. These changes include 43.5% *syntax transition* updates (e.g., "*Update router stack to HCL2*" [C5.1] Appendix A), whereas 30.4% restructuring *code block structure* of terraform related files mainly to simplify configurations and setup (e.g., "*Updated the top level terraform files with documentation. Updated variable names and defaults to match the rest of the deployment*" [C5.2] Appendix A). Finally, 26.1% of the updates are cleaning up the TF configuration code to remove unused TF configuration code (e.g., *"Basically, I went through each. tf file, cleaned up dead code, removed superfluous code comments, and made a first pass at making our terraform more idiomatic... I then updated our AWS provider to the latest v3 version."* [C5.3] Appendix A).

⑥ *Infrastructure Deployment:* In this category, updates are motivated by deployment-specific needs and make up 5.4% of the total updates. Of these, 71.4% focus on *Staging Configuration*, which typically involves preparing or adjusting Terraform configurations to support different deployment stages (i.e., development, testing, production) or environments (e.g., "*Update hashicorp/aws provider version for loadtesting and add CI validation... Update name of the new workflow"* [C6.1] Appendix A).

The remaining 28.6% aim to improve *Build Performance Optimization*, by enhancing the build execution speed or isolating build components to streamline CI/CD processes (e.g., *"Add caching to build to speed up build on subsequent runs... Change the build portion of ecs helper to be a separate task as it only needs to get called for shared infra and takes 40 odd seconds which we can shave off the other calls to terraform module... Nothing tricky here. Just a bit time consuming back and forth with the pipeline..."* [C6.2] Appendix A).

⑦ *Resource Management:* Representing 4.4% of updates, this category includes additions or changes to infrastructure resources. 35.3% of these address *Resource Dependency*, such as resolving dependency cycles or ensuring resources are used in a specific order (e.g., "*Remove SQS module as this introduced a cyclic dependency, Add explicit depends_on to resources*" [C7.1] Appendix A). Another 35.3% handle *state management*, which are provider updates related to preventing unnecessary shift or correcting drift between current TF's state and the expected infrastructure (e.g., "*These changes prevent us from deletings those tags and getting a lot of unnecessary resource changes from terraform apply*" [C7.2] Appendix A). The remaining 29.4% focus on *Resource Creation*, including provisioning new components (e.g., *"As an operator, I would like to create a downloadable script for creating s3 storage buckets"* [C7.3] Appendix A).

⑧ *Monitoring:* Monitoring-related motivations account for (2.8%) of the total updates, and mainly include updates to configure *AWS Alarms/Alerts* services (36.4%) (e.g., "*Upgrades AWS Provider to v5.53.0, Creates Metric Filter and Alarm for cloud_init_output.log when one or more startup scripts fail*" [C8.1] Appendix A). Another 36.4% of this category enables and manages *Logging* practices for AWS components (e.g., "*Add cloudwatch queries, Bump AWS provider version*" [C8.2] Appendix A). The remaining 27.3% of the category configure *Notifications* on such infrastructure components, by configuring typically through AWS Simple Notification Service (SNS), to notify developers through channels like email, Slack, or Discord (e.g., "*Add aws guard duty alerts to slack and version bump the spend alerts*" [C8.3] Appendix A).

⑨ *Performance Improvement:* This category accounts for only 2.3% of updates, where TF developers are driven by addressing cost efficiency and scalability. Within this category, sub-categories include *cache management* (44.4%), which consists of updates aimed at improving caching mechanisms to enhance load times or responsiveness (e.g., "*update s3 website to have cache times and apply to dash*" [C9.1] Appendix A). *Cost management* accounts for 33.3% of the updates in this category and includes changes made to reduce infrastructure expenses, such as avoiding unnecessary requests or leveraging more cost-effective resources (e.g., "*The goal here is mainly to prevent people from potentially driving up S3 costs by making requests we can't control or cache against the*

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

bucket" [C9.2] Appendix A). Finally, *Memory management* (22.2%) includes provider updates that allow fine-tuning of resource limits, such as enabling concurrency configuration or optimizing data transfer limits to handle larger payloads (e.g., 💬 "*upgrading provider to allow concurrency configuration*" [C9.3] Appendix A).

V. Discussions & Implications

This section discusses the implications of our findings.

A. Implications for Researchers

👉 *Examining the real-world adoption of semantic versioning in Terraform ecosystem:* RQ1 reveals a growing trend of technical lag across all studied providers between 2017 and 2025. RQ2 shows that Terraform developers more often adopt minor updates over major or patch releases. As shown in Table III, provider release activity has increased, particularly for AWS, Google, and Azure, with minor versions dominating. This suggests that maintainers prioritize incremental feature delivery, likely due to the fast-paced evolution of cloud platforms. However, this behavior contrasts with traditional dependency management [14], [15], [28], where patch releases are rapidly adopted for bug and security fixes. These trends raise concerns that some Terraform providers may not strictly follow semantic versioning [39], potentially misclassifying bug fixes as minor or major updates, leading to confusion for users. *We encourage further research on Terraform provider maintainers' adherence to SemVer, as inconsistent versioning can hinder update adoption.*

👉 *Understanding and reducing the structural complexity of provider updates:* From RQ3, pull requests (PRs) that update AWS provider dependencies exhibit different code review characteristics than PRs that do not involve such updates, particularly regarding the number of changed files, directories, commits, and code churn. These suggest that updating AWS provider dependencies introduces structural complexity and may increase the risk of unintended changes, as evidenced by a median of seven files being modified per AWS update. As a result, the effort required to manage such changes can discourage timely updates, potentially contributing to the accumulation of sub-optimal solutions. Over time, this can erode key aspects of infrastructure quality, such as maintainability and security. *In light of these findings, we encourage researchers to investigate the structural, organizational, and tooling factors that prevent infrastructure developers from keeping their configurations up-to-date. Such research could help identify recurring patterns and best practices to mitigate the complexity of updating AWS provider dependencies.*

👉 *Understanding dependency bloat in AWS Terraform providers:* The results of RQ1 and RQ2 indicate that updates to AWS provider dependencies are rarely prioritized by TF practitioners, as reflected in only 1.86% on average of TF-related commits that involve such updates. While AWS provider maintainers aim to keep the provider aligned with the evolving APIs of AWS services, TF users could rely on only a specific subset of its features tailored to their specific infrastructure needs. This mismatch suggests a phenomenon akin to *dependency bloat* [59], where the provider accumulates unused features in most real-world configurations. That is, only a tiny portion of the AWS provider is used in practice, which can lead to increased complexity, resource overhead, and hesitation to update. Prior work [60], [61] has underscored that such bloat can degrade performance and maintainability in traditional development. Therefore, *we encourage researchers to investigate empirically how TF configurations suffer from provider-level bloat. This could provide new strategies for modularizing or slimming provider releases and help the IaC community rethink how their artifacts are structured and distributed for practical use.*

B. Implications for Practitioners

👉 *Maintaining consistency across deployment environments:* Our analysis in RQ3 shows that pull requests involving AWS provider updates significantly affect more files than those without such updates. Notably, these PRs tend to span multiple directories, reflecting broader structural impact. For example, TF developers can update the AWS provider dependency in development, testing, and production directories, representing environments. Findings from RQ4 further indicate that maintaining consistency between staging and production environments is one of the motivations behind dependency updates (e.g., *"Fix Deployment of Kubernetes Tests on Jenkins"*[29]). However, aligning dependency versions across environments introduces cognitive overhead and increases maintenance effort, particularly in large, multi-directory IaC setups. *We recommend that tool builders support practitioners by offering best-practice catalogs and environment-specific configuration patterns to reduce the complexity and cost of maintaining staging environments.*

👉 *Implementing proactive notification and automated version management strategies:* From the RQ4 findings, we identified that TF developers adopt the automated solution to automatically update their AWS provider dependencies, representing 17.6% of the pull requests analyzed. However, we believe from our observation that developers are still not widely adopting automated dependency management. Therefore, *we recommend that IaC tool builders implement a more sophisticated automated solution to provide reduced alerts (i.e., to avoid alert fatigue [62]) to update AWS provider dependencies.*

👉 *Need for detecting the risk of IaC breaking changes:* From RQ1, we found that TF practitioners with AWS provider dependencies often balance between permissive versioning and restrictive constraints, unlike Azure dependencies, which favor exact pinning. This balance could introduce trade-offs between reducing technical lag and increasing the risk of breaking changes. Jafari et al. [63] assessed *permissive versioning* as a versioning smell in JavaScript, as it increases the likelihood of breaking changes due to major upgrades. In our context, this could lead to unpredictable deployments and higher maintenance costs, despite helping to reduce technical lag. In addition, backported releases further complicate dependency management. TF Practitioners may prefer older versions to avoid breaking changes while receiving fixes; however, this can lead

[29] https://tinyurl.com/y6w9ss4j

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

to an increased long-term technical lag, as illustrated in RQ1's final example. These challenges are supported by developer behavior, as observed in RQ2, where dependency constraints are rarely revised after being introduced. To mitigate these risks, *we recommend developing version monitoring tools (e.g., Docker's Watchtower[30]) that track provider updates and assess upgrade risks. These support the proactive detection of breaking changes and functional regressions.*

C. Implications for Educators

Updating course curriculum with best practices: Findings from RQ3 show that AWS provider updates affect multiple files, necessitating a substantial grasp of Terraform and the AWS services. Educators can use this in cloud computing and IaC courses to emphasize managing AWS provider dependencies, reading changelogs, and evaluating risks of outdated versions. This prepares students to professionally maintain cloud infrastructure stability and security.

VI. Threats to Validity

This section discusses threats to the validity of our study.

Internal Threats to Validity: One threat is using the GitHub API, which prior studies [64], [65] note may return non-representative or toy projects and be subject to rate limits. To address this, we collected data over multiple intervals using a file extension-based query ("*extension:tf*"), as in [2], [16], [18], to target client-side TF files. We also filter for well-engineered projects based on development activity (e.g., number of commits) and contributor count.

Construct Threats to Validity: A threat in RQ1 lies in our method to calculate technical lag. Unlike previous studies that consider the longest lag per dependency [14], [15], [54], we compute the average lag per provider type (e.g., AWS, Google) in all working directories. To reduce bias from rarely updated dependencies, we normalize by the number of directories with at least one provider. Another threat in RQ1 and RQ2 is our reliance on semantic versioning (major, minor, patch) and exclusion of pre-releases. We focus on stable versions only, as pre-releases are often instable and require explicit inclusion [13], [66]. By doing so, this follows conventions in ecosystems such as npm [14], [15], and leaves pre-release analysis for future work. In RQ3, the comparison of PRs with and without AWS updates may be affected by sampling bias. To address this, we use stratified random sampling based on the distribution of AWS update activity. For RQ4, constructing the taxonomy of update motivations may involve subjectivity. Two authors independently labeled all 390 updates, holding three review sessions to resolve disagreements and ensure consistency.

External Threats to Validity: Our analysis focuses on 194 open-source projects using AWS providers, the most consistently represented in our dataset. Future work will include a broader range of cloud providers without filtering for AWS. Although other IaC tools exist (e.g., OpenTofu [67], Terragrunt [68], Ansible [69]), we focus on Terraform due to its widespread adoption. We believe that the sample size and project diversity offer a solid basis for our analysis.

[30]https://containrrr.dev/watchtower

VII. Related Works

This section summarizes IaC research and update practices in traditional dependency management.

Infrastructure-as-Code Studies: Several studies have examined dependency management in IaC by empirically analyzing IaC artifacts and their evolution [8], [70], [71], [72]. These works provide insights into the state of practice, challenges, and gaps in managing dependencies across different IaC tools. Opdebeeck et al. [8] conducted an empirical study on *semantic versioning* adoption in Ansible roles, which are reusable components that function as shareable third-party libraries. Their findings indicate that Ansible developers adhere to semantic versioning conventions, using version increments to label different categories of modifications. Then, Drosos et al. [71] qualitatively analyzed 360 IaC-related bugs in Ansible, Puppet, and Chef. They found 23% of the bugs are related to *compatibility issues*, where an IaC program fails due to mismatches in dependencies' versions. This finding highlights the importance of dependency management in maintaining the correctness of IaC scripts. Subsequently, Opdebeeck et al. [70] investigated software supply chains of the deployment dependencies used in Ansible artifacts. They found that dependency descriptions are often informally documented in maintainer notes rather than explicitly defined within the code. This lack of formalization can lead to inconsistencies, difficulty in automating dependency resolution, and increased maintenance effort. However, their study focuses exclusively on Ansible, leaving open questions about whether similar practices exist in other IaC tools, such as Terraform and Puppet, which have different dependency management models. Recently, Begoug et al. [18] analyzed the practice of updating Terraform (TF) programs that use modules, representing a high abstraction level for provider configurations. They found that TF developers rarely prioritize module updates, with only 1.2% of commits addressing compatibility or security.

Key Difference: While prior studies offer insights into dependency management in IaC, they do not quantify how proactively developers manage provider dependencies across heterogeneous cloud infrastructures. To the best of our knowledge, this is the first study to analyze provider dependencyies updates in TF files for AWS that can be used with other platforms, examining update efforts and the factors that influence them.

Third-Party Libraries (TPLs) Update: In contemporary software development, TPLs are widely used and provide advantages, notably minimizing the work needed to implement pre-existing functionalities and services [37], [54]. TPLs undergo continuous evolution through the integration of new features and the release of new patches, which serve to address bugs present in previous versions. For practical usage of TPLs, developers must regularly update their library dependencies to the most recent versions. Previous studies have explored a variety of challenges and behaviors related to library updates, including the risks associated with outdated libraries, particularly regarding security vulnerabilities [73], [74], automated

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

approaches to support dependency update [75], [76], and developer reluctance to adopt new versions [14], [36].

Key Difference: While dependency updates have been the subject of much research in different domains, the specific traits of IaC, including its emphasis on the configuration of cloud infrastructure, its declarative approach, and the rapid pace of changes in cloud infrastructure, require a thorough examination of the dependencies of IaC [1].

VIII. Conclusion & Future Work

We studied update practices for AWS provider dependencies in 194 open-source Terraform repositories. We found a persistent technical lag, with average outdatedness reaching ∼9 months by 2025. Moreover, updates are rare, where only ≈1.86% of Terraform-related commits modified AWS provider dependencies. When updates happen via pull requests, they are large, cross-directory changes that require significant review and integration. Our thematic analysis revealed nine main drivers; the most common are dependency management, Terraform version compatibility, and security concerns. These show that updates are mostly reactive, triggered by necessity rather than proactive maintenance. These findings suggest that technical lag in IaC projects reflects the cost, complexity, and risk of updates, rather than developer negligence. Future work will focus on automated tools that recommend suitable provider versions and provide context-aware guidance to simplify updates and reduce maintenance costs.

References

[1] K. Morris, *Infrastructure as Code*. Sebastopol, CA, USA: O'Reilly Media, 2020.

[2] M. Begoug, N. Bessghaier, A. Ouni, E. A. AlOmar, and M. W. Mkaouer, "What do infrastructure-as-code practitioners discuss: An empirical study on stack overflow," in *Proc. IEEE Int. Symp. Empirical Softw. Eng. Meas.*, 2023, pp. 1–12.

[3] A. Rahman, E. Farhana, and L. Williams, "The 'as code'activities: Development anti-patterns for infrastructure as code," *Empirical Softw. Eng.*, vol. 25, no. 5, pp. 3430–3467, 2020.

[4] P. R. R. Konala, V. Kumar, and D. Bainbridge, "SoK: Static configuration analysis in infrastructure as code scripts," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience*, 2023, pp. 281–288.

[5] GitHub, "The state of open source: The 2024 octoverse report," 2024, Accessed: Apr. 10, 2025. [Online]. Available: https://github.blog/news-insights/octoverse/octoverse-2024

[6] Y. Brikman, *Terraform: Up and Running*. Sebastopol, CA, USA: O'Reilly Media, 2022.

[7] HashiCorp, "Terraform aws provider documentation," 2024. Accessed: Jul. 17, 2025. [Online]. Available: https://registry.terraform.io/providers/hashicorp/aws/latest

[8] R. Opdebeeck, A. Zerouali, C. Velázquez-Rodrıǵuez, and C. De Roover, "Does infrastructure as code adhere to semantic versioning? an analysis of ansible role evolution," in *Proc. 20th Int. Work. Conf. Source Code Anal. Manipulation*, 2020, pp. 238–248.

[9] R. Opdebeeck, A. Zerouali, C. Velázquez-Rodrıǵuez, and C. De Roover, "On the practice of semantic versioning for ansible galaxy roles: An empirical study and a change classification model," *J. Syst. Softw.*, vol. 182, 2021, Art. no. 111059.

[10] M. Begoug and A. Ouni, "How do infrastructure-as-code practitioners update their provider dependencies? an empirical study on the aws provider," in *Proc. Int. Conf. Serv.-Oriented Comput.*, 2024, pp. 373–388.

[11] "Understanding AWS provider dependency updates in infrastructure-as-code: Empirical study, taxonomy, and insights," 2025. [Online]. Available: https://figshare.com/s/6c6b0131131780c6e2c1

[12] R. Hafner, *Terraform in Depth: Infrastructure as Code With Terraform and OpenTofu*. Shelter Island, NY, USA: Manning Publications, 2025.

[13] HashiCorp, "Version constraint behavior," 2024, accessed: Oct. 23, 2024. [Online]. Available: https://tinyurl.com/5d4antb3

[14] A. Decan, T. Mens, and E. Constantinou, "On the evolution of technical lag in the npm package dependency network," in *Proc. Int. Conf. Softw. Maintenance Evol.*, 2018, pp. 404–414.

[15] A. Zerouali, E. Constantinou, T. Mens, G. Robles, and J. González-Barahona, "An empirical analysis of technical lag in npm package dependencies," in *Proc. Int. Conf. Softw. Reuse*, 2018, pp. 95–110.

[16] A. Verdet, M. Hamdaqa, L. D. Silva, and F. Khomh, "Assessing the adoption of security policies by developers in terraform across different cloud providers," *Empirical Softw. Eng.*, vol. 30, no. 3, 2025, Art. no. 74.

[17] M. Begoug, M. Chouchen, A. Ouni, E. Abdullah Alomar, and M. W. Mkaouer, "Fine-grained just-in-time defect prediction at the block level in infrastructure-as-code (IAC)," in *Proc. 21st Int. conf. Mining Softw. Repositories*, 2024, pp. 100–112.

[18] M. Begoug, A. Ouni, and M. Chouchen, "How do infrastructure-as-code practitioners update their dependencies? an empirical study on terraform module updates," in *Proc. 22nd Int. Conf. Mining Softw. Repositories*, 2025, pp. 642–653.

[19] H. Tu, Z. Yu, and T. Menzies, "Better data labelling with EMBLEM (and how that impacts defect prediction)," *IEEE Trans. Softw. Eng.*, vol. 48, no. 1, pp. 278–294, Jan. 2022.

[20] N. Munaiah, S. Kroh, C. Cabrey, and M. Nagappan, "Curating github for engineered software projects," *Empirical Softw. Eng.*, vol. 22, pp. 3219–3253, 2017.

[21] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The Promises and Perils of Mining Github," in *Proc. Int. Conf. Mining Softw. Repositories*, 2014, pp. 92–101.

[22] HashiCorp, "Terraform registry API documentation," 2024, Accessed: Mar. 22, 2025. [Online]. Available: https://developer.hashicorp.com/terraform/registry/api-docs

[23] HashiCorp, "Terraform registry," 2024. [Online]. Available: https://registry.terraform.io/

[24] M. Begoug, M. Chouchen, and A. Ouni, "Terrametrics: An open source tool for infrastructure-as-code (iac) quality metrics in terraform," in *Proc. 32nd IEEE/ACM Int. Conf. Prog. Comprehension*, 2024, pp. 450–454.

[25] A. D. Breslow, A. Tiwari, M. Schulz, L. Carrington, L. Tang, and J. Mars, "Enabling fair pricing on HPC systems with node sharing," in *Proc. Int. Conf. High Perform. Computing, Netw., Storage Anal.*, 2013, pp. 1–12.

[26] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. German, and P. Devanbu, "The promises and perils of mining git," in *Proc. Int. Conf. Mining Softw. Repositories*, 2009, pp. 1–10.

[27] H. He, R. He, H. Gu, and M. Zhou, "A large-scale empirical study on java library migrations: Prevalence, trends, and rationales," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 478–490.

[28] A. Decan and T. Mens, "What do package dependencies tell us about semantic versioning?," *IEEE Trans. Softw. Eng.*, vol. 47, no. 6, pp. 1226–1240, Jun. 2021.

[29] J. Tsay, L. Dabbish, and J. Herbsleb, "Let's talk about it: Evaluating contributions through discussion in github," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2014, pp. 144–154.

[30] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "The impact of code review coverage and code review participation on software quality: A case study of the qt, vtk, and itk projects," in *Proc. 11th Work. Conf. Mining Softw. Repositories*, 2014, pp. 192–201.

[31] G. Gousios, M. Pinzger, and A. V. Deursen, "An exploratory study of the pull-based software development model," in *Proc. 36th Int. Conf. Softw. Eng.*, 2014, pp. 345–355.

[32] X. Zhang, Y. Yu, G. Gousios, and A. Rastogi, "Pull request decisions explained: An empirical overview," *Trans. Softw. Eng.*, vol. 49, no. 2, pp. 849–871, 2022.

[33] O. Kononenko, T. Rose, O. Baysal, M. Godfrey, D. Theisen, and B. De Water, "Studying pull request merges: A case study of shopify's active merchant," in *Proc. 40th Int. Conf. Softw. Eng.: Softw. Eng. Pract.*, 2018, pp. 124–133.

[34] E. A. AlOmar, M. Chouchen, M. W. Mkaouer, and A. Ouni, "Code review practices for refactoring changes: An empirical study on openstack," in *Proc. Mining Softw. Repositories*, 2022, pp. 689–701.

[35] M. Chouchen, N. Bessghaier, M. Begoug, A. Ouni, E. Alomar, and M. W. Mkaouer, "How do software developers use chatgpt? an exploratory study on github pull requests," in *Proc. 21st Int. conf. Mining Softw. Repositories*, 2024, pp. 212–216.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.

[36] R. G. Kula, D. M. German, A. Ouni, T. Ishio, and K. Inoue, "Do developers update their library dependencies? an empirical study on the impact of security advisories on library migration," *Empirical Softw. Eng.*, vol. 23, no. 1, pp. 384–417, 2018.

[37] P. Salza, F. Palomba, D. Di Nucci, C. D'Uva, A. De Lucia, and F. Ferrucci, "Do developers update third-party libraries in mobile apps?," in *Proc. 26th Conf. Prog. Comprehension*, 2018, pp. 255–265.

[38] R. G. Kula, A. Ouni, D. M. German, and K. Inoue, "An empirical study on the impact of refactoring activities on evolving client-used apis," *Inf. Softw. Technol.*, vol. 93, pp. 186–199, 2018.

[39] Semantic versioning specification, "Semantic versioning 2.0.0," 2019. [Online]. Available: https://semver.org/

[40] A. Decan, T. Mens, A. Zerouali, and C. De Roover, "Back to the past–analysing backporting practices in package dependency networks," *IEEE Trans. Softw. Eng.*, vol. 48, no. 10, pp. 4087–4099, Oct. 2022.

[41] A. Zerouali, R. Opdebeeck, and C. De Roover, "Helm charts for kubernetes applications: Evolution, outdatedness and security risks," in *Proc. 20th Int. Conf. Mining Softw. Repositories*, 2023, pp. 523–533.

[42] HashiCorp, "Terraform aws provider changelog," 2024. Accessed: Apr. 10, 2025. [Online]. Available: https://tinyurl.com/5x8enfuk

[43] B. Gupta, P. Mittal, and T. Mufti, "A review on amazon web service (AWS), microsoft azure & Google cloud platform (GCP) services," in *Proc. 2nd Int. Conf. ICT Digit., Smart, Sustain. Develop.*, 2021, Art. no. 9.

[44] P. Pooput and P. Muenchaisri, "Finding impact factors for rejection of pull requests on github," in *Proc. Int. conf. Netw., Commun. Comput.*, 2018, pp. 70–76.

[45] M. Caulo, B. Lin, G. Bavota, G. Scanniello, and M. Lanza, "Knowledge transfer in modern code review," in *Proc. 28th Int. Conf. Prog. Comprehension*, 2020, pp. 230–240.

[46] J. W. Tukey et al., *Exploratory Data Analysis*, vol. 2. Berlin, Germany: Springer, 1977.

[47] S. S. Shapiro and M. B. Wilk, "An analysis of variance test for normality (complete samples)," *Biometrika*, vol. 52, no. 3/4, pp. 591–611, 1965.

[48] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, pp. 50–60, 1947.

[49] J. Romano, J. D. Kromrey, J. Coraggio, and J. Skowronek, "Appropriate statistics for ordinal level data: Should we really be using t-test and cohen'sd for evaluating group differences on the NSSE and other surveys," in *Proc. Annu. Meeting Florida Assoc. Institutional Res.*, 2006, Art. no. 34.

[50] N. Davila and I. Nunes, "A systematic literature review and taxonomy of modern code review," *J. Syst. Softw.*, vol. 177, 2021, Art. no. 110951.

[51] M. Chouchen, A. Ouni, and M. W. Mkaouer, "Multicr: Predicting merged and abandoned code changes in modern code review using multi-objective search," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–44, 2024.

[52] O. Kononenko, O. Baysal, L. Guerrouj, Y. Cao, and M. W. Godfrey, "Investigating code review quality: Do people and participation matter?," in *Proc. Int. Conf. Softw. Maintenance Evol.*, 2015, pp. 111–120.

[53] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "An empirical study of the impact of modern code review practices on software quality," *Empirical Softw. Eng.*, vol. 21, no. 5, pp. 2146–2189, 2016.

[54] P. Salza, F. Palomba, D. Di Nucci, A. De Lucia, and F. Ferrucci, "Third-party libraries in mobile apps: When, how, and why developers update them," *Empirical Softw. Eng.*, vol. 25, no. 3, pp. 2341–2377, 2020.

[55] H. Alrubaye, M. W. Mkaouer, and A. Ouni, "Migrationminer: An automated detection tool of third-party java library migration at the method level," in *Proc. Int. Conf. Softw. Maintenance Evol.*, 2019, pp. 414–417.

[56] V. Braun and V. Clarke, *Thematic Analysis*. Washington, DC, USA: American Psychological Association, 2012.

[57] D. S. Cruzes and T. Dyba, "Recommended steps for thematic synthesis in software engineering," in *Proc. IEEE Int. Symp. Empirical Softw. Eng. Meas.*, 2011, pp. 275–284.

[58] M. L. McHugh, "Interrater reliability: The kappa statistic," *Biochemia Medica*, vol. 22, no. 3, pp. 276–282, 2012.

[59] G. Xu, N. Mitchell, M. Arnold, A. Rountev, and G. Sevitsky, "Software bloat analysis: Finding, removing, and preventing performance problems in modern large-scale object-oriented applications," in *Proc. FSE/SDP Workshop Future Softw. Eng. Res.*, 2010, pp. 421–426.

[60] G.-P. Drosos, T. Sotiropoulos, D. Spinellis, and D. Mitropoulos, "Bloat beneath python's scales: A fine-grained inter-project dependency analysis," *Proc. ACM on Softw. Eng.*, vol. 1, no. FSE, pp. 2584–2607, 2024.

[61] C. Soto-Valero, T. Durieux, and B. Baudry, "A longitudinal analysis of bloated java dependencies," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 1021–1031.

[62] R. G. Kula, "Reducing alert fatigue via ai-assisted negotiation: A case for dependabot," 2025, *arXiv:2502.06175*.

[63] A. J. Jafari, D. E. Costa, R. Abdalkareem, E. Shihab, and N. Tsantalis, "Dependency smells in javascript projects," *Trans. Softw. Eng.*, vol. 48, no. 10, pp. 3790–3807, 2021.

[64] O. Dabic, E. Aghajani, and G. Bavota, "Sampling projects in github for MSR studies," in *Proc. 18th Int. Conf. Mining Softw. Repositories*, 2021, pp. 560–564.

[65] V. Cosentino, J. Luis, and J. Cabot, "Findings from github: Methods, datasets and limitations," in *Proc. 13th Int. Conf. Mining Softw. Repositories*, 2016, pp. 137–141.

[66] HashiCorp, "Error thrown on invalid provider address during terraform init," gitHub issue comment, 2022. [Online]. Available: https://tinyurl.com/2vkht32a

[67] "Opentofu," 2023. Accessed: Nov. 08, 2024. [Online]. Available: https://opentofu.org/

[68] "Terragrunt documentation," 2016. Accessed: Nov. 08, 2024. [Online]. Available: https://terragrunt.gruntwork.io/

[69] "Ansible by red hat," 2012. Accessed: Sep. 13, 2024. [Online]. Available: https://www.ansible.com/

[70] R. Opdebeeck, B. Adams, and C. De Roover, "Analysing software supply chains of infrastructure as code: Extraction of ansible plugin dependencies," in *Proc. Int. Conf. Softw. Anal., Evol. Reengineering*, 2025, pp. 181–192.

[71] G.-P. Drosos, T. Sotiropoulos, G. Alexopoulos, D. Mitropoulos, and Z. Su, "When your infrastructure is a buggy program: Understanding faults in infrastructure as code ecosystems," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, pp. 2490–2520, 2024.

[72] P. T. Kon et al., "Iac-eval: A code generation benchmark for cloud infrastructure-as-code programs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 134488–134506.

[73] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the NPM ecosystem," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 995–1010.

[74] K. Lim, Y. Kwon, and D. Kim, "A longitudinal study of vulnerable client-side resources and web developers' updating behaviors," in *Proc. ACM Internet Meas. Conf.*, 2023, pp. 162–180.

[75] J. Dietrich, D. Pearce, J. Stringer, A. Tahir, and K. Blincoe, "Dependency versioning in the wild," in *Proc. 16th Int. Conf. Mining Softw. Repositories*, 2019, pp. 349–359.

[76] L. Lin, J. Ye, C. Wang, and R. Wu, "PATEN: Identifying unpatched third-party apis via fine-grained patch-enhanced AST-level signature," *IEEE Trans. Softw. Eng.*, vol. 51, no. 4, pp. 990–1006, Apr. 2025.

Mahi Begoug received the master's degree from the University of Montpellier, France, in 2022, and an engineering degree in computer science from the École Supérieure d'Informatique (ESI, formerly INI) in Algiers, Algeria, in 2021. He is currently working toward the PhD degree in the Department of Software Engineering and IT, ÉTS Montreal, University of Quebec, which he joined, in 2023. His research interests include software maintenance, infrastructure as code, and empirical software engineering.

Ali Ouni received the PhD degree in computer science from the University of Montreal. He is currently a full professor with the Department of Software Engineering and IT, ETS Montreal, University of Quebec, where he leads the Software Technology and Intelligence (STIL) Research Lab. His research interests are in the intersection between software engineering and artificial intelligence, including software maintenance and evolution, refactoring of software systems, software quality, service-oriented computing, and the application of artificial intelligence techniques to software engineering. He was a program committee member and reviewer in several journals and conferences. He is a member of the ACM and IEEE Computer Society.

Jasem Khelifi is currently working toward the PhD degree in the Department of Software Engineering and IT at ÉTS Montreal, University of Quebec, where he is a member of the STIL research lab. His research interests include continuous integration practices in microservices, software gamification, and GitHub Actions. He received his engineering degree in software engineering from South Mediterranean University (MedTech), Tunisia.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 23,2026 at 23:14:50 UTC from IEEE Xplore. Restrictions apply.